Comparing Scientific Machine Learning Methods for Nonlinear ODE Dynamics

Xuanyou Liu (NetID: XLM6838)

ELE_ENG 395 Scientific Machine Learning, Final Project, March 13, 2026

Abstract. This project compares data-driven and physics-structured models for learning nonlinear ODE dynamics from noisy trajectories. I evaluate baseline neural networks, Neural ODE rollouts, and a structured Lagrangian neural network (LNN) on two systems from the course: Van der Pol and the simple pendulum. The main finding is that model structure and training setup strongly affect performance: on pendulum, the LNN is substantially more accurate than black-box baselines; on one-trajectory Van der Pol fitting, direct regression can outperform long-horizon rollout training.

1. Introduction

Learning dynamics from noisy trajectories is central in scientific machine learning. Black-box regressors can fit data quickly but may violate dynamical consistency; rollout models are physically interpretable but can accumulate error; physics-structured models can exploit known invariances. This project extends Chapter 5 and Chapter 7 homework into a unified benchmark with reproducible code, common metrics, and direct model comparison.

Objective. Compare baseline MLPs, Neural ODEs, and a structured LNN in terms of trajectory RMSE, phase-space behavior, and training cost.

2. Method

Van der Pol. I simulated

$$\dot{z} = v, \qquad \dot{v} = \mu(1 - z^2)v - z, \quad \mu = 1$$

using RK4 on $t \in [0, 20]$ with $\Delta t = 0.1$, then added Gaussian noise with $\sigma = 0.1$.

Pendulum. I simulated

$$\ddot{\theta} + \frac{g}{\ell}\sin\theta = 0, \quad \frac{g}{\ell} = 9.81,$$

with 80 RK4 trajectories on $t \in [0, 10]$ (400 points, $\Delta t \approx 0.0251$), noise $\sigma = 0.02$, and trajectory split 56/12/12 (train/val/test).

Models. (1) *Baseline MLP*: Van der Pol uses $t \mapsto z(t)$; pendulum uses one-step map $h_n \mapsto h_{n+1}$ with $h = (\theta, \omega)$ and autoregressive rollout. (2) *Neural ODE*: learn vector field $F_\theta : \mathbb{R}^2 \to \mathbb{R}^2$ and roll out

$$h_{n+1} = h_n + \Delta t\, F_\theta(h_n)$$

with trajectory MSE loss. (3) *Structured LNN (pendulum)*: use

$$L_W(\theta, \omega) = \frac{1}{2}\omega^2 - V_W(\theta), \qquad \ddot{\theta}_{\text{pred}} = -\frac{\mathrm{d}V_W}{\mathrm{d}\theta},$$

train on finite-difference acceleration targets.

3. Results

Table 1: Trajectory RMSE comparison.

System	Model	State 1 RMSE	State 2 RMSE
Van der Pol	Baseline MLP	0.0533 (z)	0.2392 (v)
Van der Pol	Neural ODE	0.1012 (z)	0.3050 (v)
Pendulum	Baseline MLP	0.6351 (θ)	1.9294 (ω)
Pendulum	Neural ODE	0.3819 (θ)	1.1534 (ω)
Pendulum	Structured LNN	0.0067 (θ)	0.0204 (ω)

Capacity and spectral-bias observation. For Van der Pol, the base 2x32 MLP performed best; the deep 6x32 model was unstable and underfit oscillatory details. This is consistent with spectral-bias intuition and optimization sensitivity: deeper networks did not automatically improve high-frequency fitting in this setup.

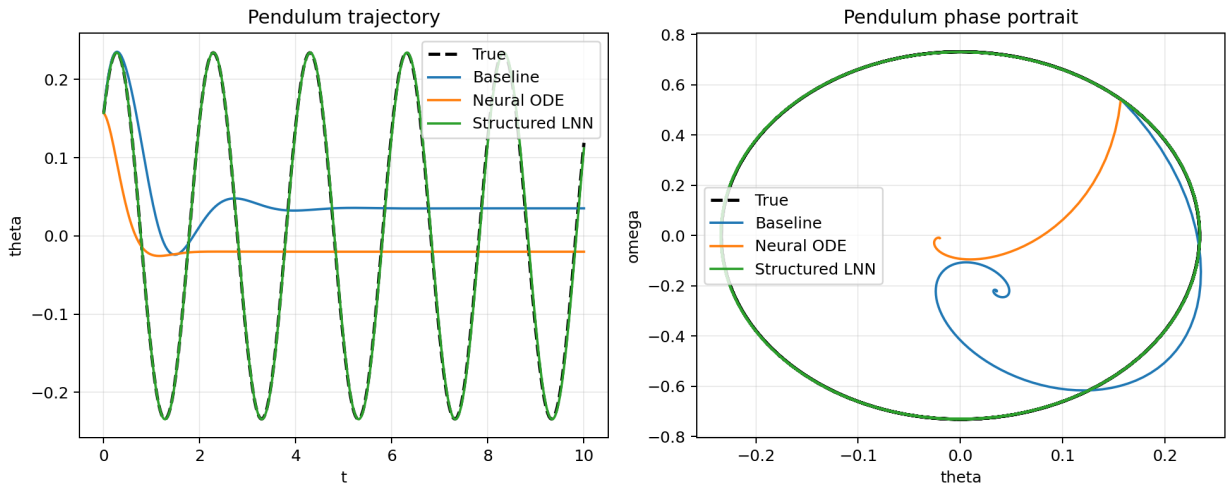

Figure 1: Representative pendulum test trajectory and phase portrait. The structured LNN tracks true dynamics closely, while baseline and Neural ODE drift.

4. Discussion

Two clear conclusions emerge. First, architecture complexity alone is not enough; training objective and rollout stability matter. On single-trajectory Van der Pol fitting, direct regression was more accurate than Neural ODE rollout because integration errors compound over horizon. Second, when correct physics structure is available, inductive bias is powerful: the pendulum LNN reduced RMSE by nearly two orders of magnitude relative to black-box baselines.

Limitations. The Van der Pol comparison uses one primary initial condition and first-order Euler rollout for Neural ODE training. Both choices can disadvantage rollout models.

Future work. Use higher-order differentiable integrators for Neural ODE training, evaluate across larger initial-condition distributions, and add PINN-style residual constraints.

Reproducibility. All experiments are run by `pythoncode/scripts/run_experiments.py`; metrics are exported to `code/results_summary.json`.